



# Comparing Four Important Sorting Algorithms Based on Their Time Complexity

Soheil Sepahyar<sup>†</sup>  
Computer Science  
Michigan Technological  
University  
Houghton Michigan USA  
sepahyar@mtu.edu

Reza Vaziri  
Computer Engineering  
Islamic Azad University of Central  
Tehran Branch  
Tehran, Iran  
rez.vaziri@iauctb.ac.ir

Marzieh Rezaei  
Computer Engineering  
Islamic Azad University of Central  
Tehran Branch  
Tehran, Iran  
marziehrezai930@gmail.com

## ABSTRACT

Time complexity and memory complexity are significant for all algorithms, especially sorting algorithms. Using the right sorting algorithm for our data can possibly decrease time and memory usage. The sorting problem has attracted a great deal of attention because efficient sorting is essential to optimize other algorithms as well. Most of the time, a sorting algorithm consists of two nested loops, which can determine the complexity of the algorithm; however, other factors such as the number of data and data types play an important role as well. Thus, by using the right sorting algorithm, we can make more efficient use of time and memory. In this paper, we use different sorting algorithms for different data types in order to determine the optimum use of time and memory for these algorithms. It means that if we know what kind of dataset we have, it can help us to use a more efficient algorithm even close to linear time. One of the interesting results is shell sort. So, by checking each type of data, like primes, Fibonacci, odd, and even datasets, we can know more information about each sorting algorithm. Also, we can determine which algorithm needs additional memory for sorting and which does not. To reduce the time and memory allocation, we use various data, such as numbers in random order or reverse order, as well as a large number of data and a small number of data. By comparing the results, the optimal algorithm in various scenarios may be recognized.

## KEYWORDS

Time Complexity, Sorting, Quick Sort, Shell Sort, Insertion Sort, Selection Sort, Fibonacci, Prime, Odd, Even

## CCS CONCEPTS

•Theory of computation~ Design and analysis of algorithms~ Data structures design and analysis~ Sorting and searching

<sup>†</sup>Corresponding author: sepahyar@mtu.edu

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

ACAI '19, December 20–22, 2019, Sanya, China

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-7261-9/19/12...\$15.00

<https://doi.org/10.1145/3377713.3377808>

•Theory of computation~ Design and analysis of algorithms~ Algorithm design techniques

## ACM Reference format:

Soheil Sepahyar, Reza Vaziri and Marzieh Rezaei. 2019. Comparing Four Important Sorting Algorithms Based on Their Time Complexity. In *Proceedings of 2019 2nd International Conference on Algorithms, Computing and Artificial Intelligence (ACAI'19)*. Sanya, China, 8 pages. <https://doi.org/10.1145/3377713.3377808>

## 1 Introduction

In computer science, the time complexity of an algorithm quantifies the amount of time taken by an algorithm to run as a function of the length of the string representing the input. [1] Sorting is ordering a list of objects. It is important how to use an algorithm with the type of data (for example long, integer or string and others). The time complexity of an algorithm is commonly expressed using big O notation, which excludes coefficients and lowers order terms. The Big O notation is used to classify algorithms by how they respond to changes input size. Knowing how and when to use a particular algorithm could give us a better understanding of the performance. Graphical comparison of sorting algorithms can provide us with a better picture of time complexity. In this paper, we focus on common sorting. We calculate the average case or the worst case in each algorithm. [2] Some of the sorting algorithms are sophisticated to implement like Quicksort, but they have the best time complexity (like  $n$  or  $n \log n$ ) in some conditions that we can distinguish them by our experiment research. Also, each sorting algorithm has its methodology in execution. We also check some other behavior of sorting algorithms like stability or In-Place and Out-Place. In this paper, we focus on four algorithms:

- Insertion Sort
- Shell Sort
- Quick Sort
- Selection Sort
- 

We compare them with different data size and classify them based on some properties:

- Time Complexity.

- Data size (the size of numbers that we generated them for sorting)
- Stability
- In-Place or Out-Place

There are two versions of sorting. The first one is Internal Sorting and the second one is External Sorting. The Internal Sorting is a type of sorting in which algorithms can handle massive amounts of data. The External Sorting is required when sorted data does not fit in the main memory (most of the time a RAM) and to solve this problem they usually use the external hard drive (auxiliary memory) [3]. In this paper, we focus on Internal Sorting algorithms. Also, in reference [4], they classify algorithms based on their types and owing to this fact, for the implementation of our algorithms we have two types. The first one is Incremental and the second one is divide and conquer. It seems that some papers have a deficiency, which is sorting ascending data number, although there is no need for sorting this kind of numerical data [2, 5, 6]. However, the focus of this paper is on the random number for the average case and descending number for the worst case.

## 2 Computation and Analysis

### 2.1 Programming

For calculating our time complexities, we generate some data in different ways such as making massive numbers in random or ascending numbers, write the program with C++ language and use dynamic arrays for generated numbers. With calculation of each algorithm, we get their time processing. The process of each algorithm will help us to understand the best case of each algorithm and have efficiency in our process later. The arrays were deleted at the end of program processing to release the physical memory. Moreover, we processed this experiment by the following computer specification:

**Physical Memory (RAM): 16 GB DDR4**

**CPU: Intel Core i7 6700HQ**

**Thread Numbers: 8**

**Core Numbers: 4**

Also, datatype affects time complexity. For example, String needs more time for running in comparison with Integer data type [7]. The focus of this paper is Integer datatype.

### 2.2 Stability

A sorting algorithm is stable if two objects with equal keys appear in the same order in sorted output as they appear in the unsorted input array. Some sorting algorithms are stable by nature, and some sorting algorithms are not. When same elements are indistinguishable, such as integers or more generally, any data where the entire part is the key, stability is not an issue. Stability is also not an issue if all keys are different. In general, any comparison based on sorting algorithm which is not stable by nature can be modified to be stable by changing the key comparison operation so that the comparison of two keys considers the position as a factor for objects with equal keys [8]. In another word, a sorting method generating the permutation  $\sigma$  will be called stable [9].

### 2.3 In-Place Out-Place Algorithms

In computer science, if we use an algorithm that transforms input no auxiliary data structure, we use an In-Place algorithm. In-Place algorithm updates data sequence only through replacement or swapping of elements. An algorithm, which is not In-Place, is out-of-place. [4]

### 2.4 Data

Our program generates data. In this paper we use four types of data in quantity:

1. Large Random Dataset.
2. Large Reverse Dataset.

In addition, we knew we do not need to generate ascending numbers because it is already sorted and we do not need to calculate their time complexity.

### 2.5 Timer Function

The "Start Counter" and "Get Counter" are two functions that can calculate the process time of each algorithm, and we call them in each algorithm in our C++ code. (Also, these two functions were written by other references that had written in Stack Overflow. For more information, you can check the references.) For the Start Counter function we have:

```
double PCFreq = 0.0;
__int64 CounterStart = 0;
void StartCounter() {
    LARGE_INTEGER li;
    if (!QueryPerformanceFrequency(&li))
        cout << "QueryPerformanceFrequency failed!\n";
    PCFreq = double(li.QuadPart) / 1000.0;
    QueryPerformanceCounter(&li);
    CounterStart = li.QuadPart;
}
```

For the GetCounter we have :

```
double
GetCounter() {
    LARGE_INTEGER li;
    QueryPerformanceCounter(&li);
    return double(li.QuadPart - CounterStart) / PCFreq; }
```

With these functions, we can measure the complete time of the particular algorithm.

## 3 Algorithm

Each algorithm has its way of sorting numbers. We use four types of conventional algorithms.

### 3.1 Insertion Sort

Insertion sort is a simple sorting algorithm. In Insertion sort, we have two loops, and we know that most of the time if an algorithm consists of two loops its time complexity is  $O(n^2)$  [8]. When we insert a new element, we find its target location and slide each item after this place ahead by one array position to make room for the new insertion [10]. Insertion sort is so fast for increasing numbers, and each value is compared only once and does not need to move beyond any other value. The source code of this algorithm that we use in our program for execution is:

```
void insertionsort(int *x, int n) {
```

```

int temp;
for (int i = 1; i < n; i++) {
    for (int j = i; j >= 1; j--) {
        if (x[j] < x[j - 1]) {
            temp = x[j]; x[j] = x[j - 1]; x[j - 1] = temp; } else
            break; } }

```

This program swaps the numbers to sort the list simply. The pointer (x) which we use, belongs to our dynamic array that holds generated numbers inside it. Furthermore, we know that insertion sort is stable sort and In-Place. It means this algorithm does not need extra memory (new array) to sort the numbers. For the implementation of execution (Or executive) function, we use a dynamic array with pointer:

```

for (n = 100; n < 100000000; n = n * 2) {
    A = new int[n];
    for (int k = 0; k < n; k++) {
        A[k] = rand(); }
    StartCounter();
    insertionSort(A, n);
    endTime = GetCounter();
    delete[] A;
    A = NULL; // free the memory }

```

The above function is for running the algorithm in random mode. For running other three sorting algorithms, we use the same way of above function. In other data sizes, we change the random mode to decreasing number.

```

for (int k = 0; k < n; k++)
{ A[k] = rand(); }
Change to int n = 1000;
for (int k = 0; k < n; k++) { A[k] = n;
n -= rand(); }

```

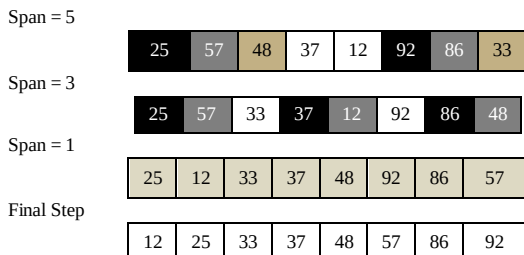
Also, we could use a clock\_t variable in C++ language, but with using these time functions, we have accurate time.

### 3.2 Shell Sort

Shellsort, also known as Shell sort or Shell's method, is an In-Place comparison sort. The method starts by sorting pairs of elements far apart from each other, then progressively reducing the gap between elements to be compared. The process starts with far apart elements. Donald Shell published the first version of this sort in 1959 [11]. These steps are used for shell sort:

- Step 1. Initialize the value of h**
- Step 2. Divide the list into smaller sub list of equal interval h**
- Step 3. Sort these sub-lists using insertion sort**
- Step 4. Repeat until complete list is sorted**

In Figure 1 the example of this algorithm has shown.



**Figure 1: Shell sort algorithm example, Step by Step.**

The source code of shell sort is:

```

void shell_sort_first_gap(int array[], int n) {
    static int gap_sequence[] = {13, 9, 5, 3, 2, 1};
    for (int gap : gap_sequence)
        if (gap < n) {
            for (int i = gap; i < n; ++i)
                for (int j = i - gap; j >= 0 && array[j] > array[j + gap]; j -= gap)
                    swap(array[j], array[j + gap]); } }

```

In this algorithm, we can sort numbers in each step by insertion sort. Prime numbers are the best choices for gap sequence. For the final step of span = 1, most of the elements have been already sorted. Also for Shellsort, Pratt [12] shows that using a sequence consisting of all products of powers of two and three less than n results in a worst-case running time of  $O(n \log^2 n)$  as we show it in Table 1. In [11] a new research has been shown if this algorithm is randomized, its time complexity is  $O(n \log n)$  and by giving any input with very high probability.

For this algorithm, we use four different gap sequences:

- I. [55, 34, 21, 13, 8, 5, 3, 2, 1]
- II. [23, 19, 17, 13, 11, 7, 5, 3, 1]
- III. [18, 16, 14, 12, 10, 8, 4, 2, 1]
- IV. [17, 15, 13, 11, 9, 7, 5, 3, 1]
1. Fibonacci Numbers
2. Prime Numbers
3. Even Numbers
4. Odd Numbers

As is illustrated in Shellsort graphs in figures 9 and 10, the result of these four gap sequences will be different in time complexity.

**Table 1: Sorting Algorithms Time Complexities**

| SORTING ALGORITHM | BEST CASE     | AVERAGE CASE                     | WORST CASE      | STABILITY | IN-PLACE OR OUTPLACE |
|-------------------|---------------|----------------------------------|-----------------|-----------|----------------------|
| INSERTION SORT    | $O(n)$        | $O(n^2)$                         | $O(n^2)$        | Yes       | In-Place             |
| SHELL SORT        | $O(n \log n)$ | depends on gap sequence and data | $O(n \log^2 n)$ | No        | In-Place             |
| QUICK SORT        | $O(n \log n)$ | $O(n \log n)$                    | $O(n^2)$        | No        | In-Place             |
| SELECTION SORT    | $O(n^2)$      | $O(n^2)$                         | $O(n^2)$        | No        | In-Place             |

### 3.3 Quick Sort

Quick Sort is one of the algorithms that uses divide and conquer method. This algorithm was created in 1962 by C.A.R Hoare [13]. It is one of the best algorithms in a large amount of data in sorting and It has been analyzed and studied extensively in [14, 15 16]. Meanwhile, Quicksort is the standard sorting algorithm that has been used hitherto. It is an expedient sorting algorithm in random mode. In Quicksort algorithm, we specify a pivot number in some situations. One of the common conditions is using a random number for pivot or making an average of highest and lowest data. The efficiency of choosing the pivot number depends on our data in some cases, but we use pivot number by calculating the average of our highest and lowest number. It is an In-Place algorithm, and it does not need any extra memory for sorting. Moreover, it's not

stable sorting algorithm [13, 17]. In Quicksort the average case is  $O(n \log n)$ , and the worst case is  $O(n^2)$  as the proof of this results are in Horowitz, Ellis, and Sartaj Sahni. Fundamentals of computer algorithms [18], but in this experiment, we calculate it again and test this algorithm with different size of data. The source code of Quicksort algorithm in our research is:

```
void Quicksort(int *x, int low, int high) {
    int i = low;
    int j = high;
    int tmp;
    int pivot = x[(low + high) / 2];
    while (i <= j) {
        while (x[i] < pivot)
            i++;
        while (x[j] > pivot)
            j--;
        if (i <= j) {
            tmp = x[i];
            x[i] = x[j];
            x[j] = tmp;
            i++;
            j--;
        }
    }
    if (low < j)
        Quicksort(x, low, j); // The recursive call
    if (high > i)
        Quicksort(x, i, high);
}
```

After this code, we execute them in the similar way of other algorithm execution. We execute it in the random large amount of data, and the highest to lowest (descending). For large datasets, Quicksort gives a better performance in most of the examples in comparison with other algorithms [19]. Also, we use a different mode of pivot numbers for these sorting algorithms. Choosing the pivot number can affect our time complexity in Quicksort algorithm.

We implemented four types of pivot number:

1. The median (middle) element between low and high.
2. Random element between low and high.
3. Average of first three elements between low and high.
4. Average of three random elements between low and high.

### 3.4 Selection Sort

Selection sort is an In-Place algorithm, and it is not the stable algorithm. However, we can implement selection sort as the stable sorting algorithm in our code. It is not an efficient and fast sorting algorithm and its time complexity is  $O(n^2)$ . This algorithm is worst with a large dataset. Most of the time, Insertion sort is faster than Selection sort. It is one of the easiest and most powerful sorting algorithms for working with a small amount of dataset. If your data has small keys but large data area, selection sort may be the quickest [20]. This algorithm sustains two sub-arrays in a given array:

1. The sub-array, which has been already sorted.
2. Remaining sub-array, which is unsorted. Furthermore, Selection sort is not difficult to analyze in comparison with other sorting algorithms since none of the loops depend on the data in the array. [5, 21].

The code of this algorithm is:

```
void selectionSort(int *x, int n) {
```

```
    for (int i = 0; i < n - 1; i++) {
        pos_min = i;
        for (int j = i + 1; j < n; j++) {
            if (x[j] < x[pos_min])
                pos_min = j;
        }
        if (pos_min != i)
            { temp = x[i]; x[i] = x[pos_min]; x[pos_min] = temp; }
    }
```

## 4 Execution

For running our source code after implementation, we have an option to choose the algorithms.

In Figure 2 the first column is the quantity of generated numbers and the second one is the time that each number of data take to be sorted ( $T_n$ ). In this program, we use set precision in C++ to have the accurate time.



| Enter the number of choice you want : 1 | All the time is :     |
|-----------------------------------------|-----------------------|
| 10                                      | 0.002370371306813356  |
| 20                                      | 0.0027654331912822486 |
| 40                                      | 0.0082962995738467462 |
| 80                                      | 0.0355556960220034    |
| 160                                     | 0.10113584242403052   |
| 320                                     | 0.25600010113584243   |
| 640                                     | 0.80197562547185208   |
| 1280                                    | 2.3134823954498356    |
| 2560                                    | 9.067065310445555     |
| 5120                                    | 34.968507641879562    |
| 10240                                   | 134.13378138618526    |

**Figure 2: After running the program and selecting one of the algorithms**

## 5 Computation and Results

For analyzing the graphs, we start with Figure 3 in which Selection sort has been shown. As it was declared in Table 1 the time complexity of this sorting algorithm in both random and reverse mode is  $O(n^2)$ . In this sorting algorithm, the random mode is more complex than its reverse mode because of being more fluctuant than the reverse mode. However, after large dataset, as it has shown after 80000-dataset size the differences appear. In Figure number 4 we see Shellsort Fibonacci Type as it was expected. The reverse mode becomes so complex and higher than its random mode. The average case depends on random numbers that have been generated but we should conclude that for the reverse mode the time complexity is  $O(n \log_2^2 n)$  which is more complex than the average mode. The differences appear after processing of 4000 number of the dataset. Random numbers generated by the computer may be already sorted by this sorting algorithm. This helps algorithm to sort in less time than it takes in random mode.

In Figure number 5 Quicksort Median Pivot Number has been shown, and we see that the time complexity is  $O(n \log n)$  as it was expected. The differences in reverse mode and random mode are so clear, and the reverse mode is more complex than random because there are more variations (or fluctuations) in reverse mode in comparison with random mode. Also, we see that our datasets in this algorithm are more than other algorithms. For Quicksort algorithm with a large number of datasets, this sorting algorithm can have a better result in a large number of datasets. Moreover, because of using an array of our generated elements, the Quicksort is a good option rather than another sort like Mergesort in a large number of datasets. Because of locality of reference requiring extra memory for data sorting, Mergesort might be slow for numerous datasets.

In Figure number 6 Insertion Sort time complexity is  $O(n^2)$  as it was predicted in our Table 1. It is less efficient in a large number of datasets than advanced algorithms such as Quicksort and Mergesort. As we see Insertion sort sorts a smaller number of datasets in  $O(n^2)$  while we can sort more number of datasets in less time complexity by Quicksort as we have shown it in figures number 5, 7 and 8. Although Insertion sort has some problems in a large number of datasets. But for the small number of datasets, it is more efficient than other algorithms. Also, it is a stable algorithm. We also calculate four types of Quicksort algorithms and four types of Shellsort algorithms together. For Quicksort algorithm, the main difference is the way that we choose pivot number. As we see in Figure 7, for large datasets of Quicksort reverse mode there are little differences between four types. Quicksort first type has the best results in comparison with other three types. For the best situation as the first type, we use a middle number between our datasets. It gives us the best situation to choose our pivot number. If the list is partially ordered, the central (or middle) element as a pivot number in the partition will have the best result. Worst-case behavior occurs when the central element happens to be the largest or smallest element each time partition is invoked. As is shown in Quicksort type 2 and type 4. In these types, the pivot number is the smallest and largest element of our array.

For large datasets in Quicksort Random and Reverse mode, the same situation happens. The best situation is the pivot number as a median number of our array datasets, and the worst situation is first three average elements or highest number or random mode that has been chosen.

Figure 9 and 10 compare four Types of Shellsort with different gap size. As mentioned above in the Shellsort section, the first type belongs to random numbers, the second type is based on prime numbers, the third type pertains to even numbers, and the fourth type is related to odd numbers for gap size. The interesting thing about Figures 9 and 10 is that the best time complexity with a massive difference belongs to Shellsort with Fibonacci gap-sequence (Type I). This shows that the Fibonacci numbers can make Shellsort algorithm more efficient than the other types Even Prime numbers and after Fibonacci gap-sequence, prime numbers are the best case for gap-sequences. However, there are some limitations in this algorithm. If gap size becomes larger, there is no guaranty that algorithm remains efficient. In fact, there is a limitation in numbers of gap sizes for Shellsort. Ten or Nine gap sequences can have an effective result in the Shellsort algorithm as is shown in Figure 9 and 10. The worst case belongs to type 4 with odd numbers. For the Shellsort algorithm, odd numbers are

the worst numbers for being gap sequences of this algorithm. In the opposite, prime numbers as a gap sequences are one of the best choices for the Shellsort algorithm. Hence, by choosing the suitable and proper gap sequences for the Shellsort algorithm, we would have an effective sorting algorithm for our data. The worst case of Shellsort is Insertion sort. By having different gap sizes, we can change its time complexity. Although gap sequences can affect our time complexity strongly, we cannot prove that all the gap sequences work statically. In fact, there is no way to prove especial gap sequences will work the best case or worst case all the time. With this experiment, we see that Fibonacci gap sequences work as the best case.

For the Final Step, we compare three sorting algorithms (Shellsort, Selection Sort, and Insertion Sort except for Quicksort) in the above graph. Because Quick Sort is faster than all the above sorting algorithms and differences are very massive. As it was shown in the above graph, Shellsort best case which is Fibonacci gap-sequences is better than Selection Sort and Insertion Sort. One of the important reasons that causes this difference with Fibonacci gap-sequences is Fibonacci sequence has a very fast growth over the rest of the sequences such as prime numbers and it will work better in large amounts of data. As it was expected, Insertion Sort is worse than Shellsort because Shellsort as the worst case is Insertion Sort. So Insertion Sort should be worse than Shell's algorithm. For Selection sort, we have Time complexity of  $O(n^2)$  all the time. That is the reason why it is the worst case in the above graphs. Also, all of four sorting algorithms' Time Complexities are formulated in Table 1. As it was expected, the best result with less time complexity and  $O(n \log n)$  belongs to Quicksort Type 1 (Median Pivot), and the worst case pertains to Shellsort Type 4 (Odd numbers for gap-sequences). For other algorithms, the result is the same as it was expected. Other algorithms are better and more efficient (effective) in fewer numbers of datasets such as Insertion sort and Selection sort. Also, the other way that helps for Quicksort algorithm to become more efficient is having more pivot numbers than one pivot number, and we will research on this problem in our future work.



Figure 3: Selection Sort Dataset

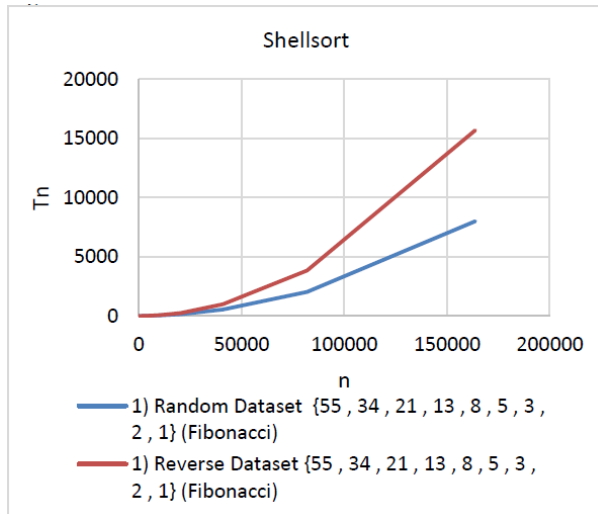


Figure 4: Shellsort Fibonacci Dataset (Best Case)

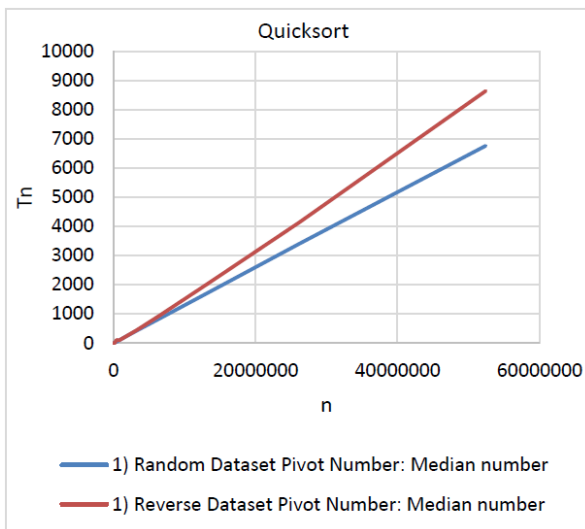


Figure 5: Quick Sort Median Pivot Number

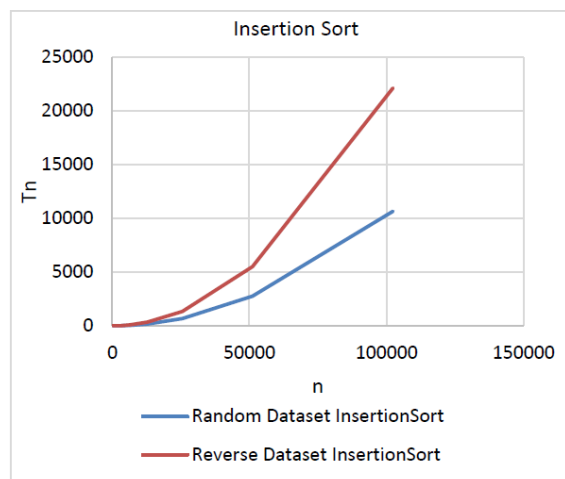


Figure 6: Insertion Sort Dataset

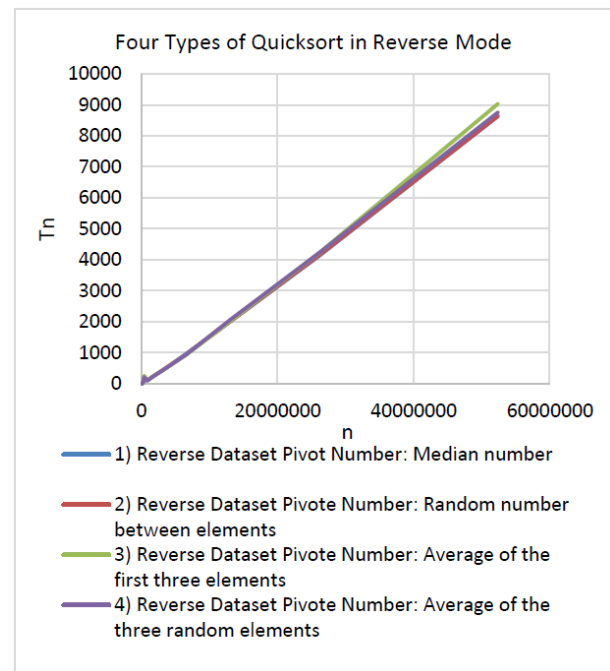


Figure 7: Four Types of Quicksort Algorithm in Reverse Mode.

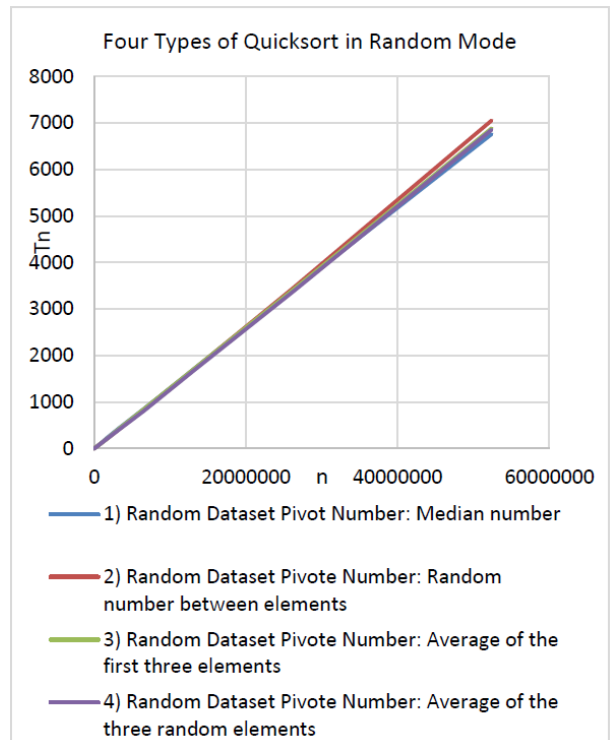


Figure 8: Four Types of Quicksort Algorithm in Random Mode.



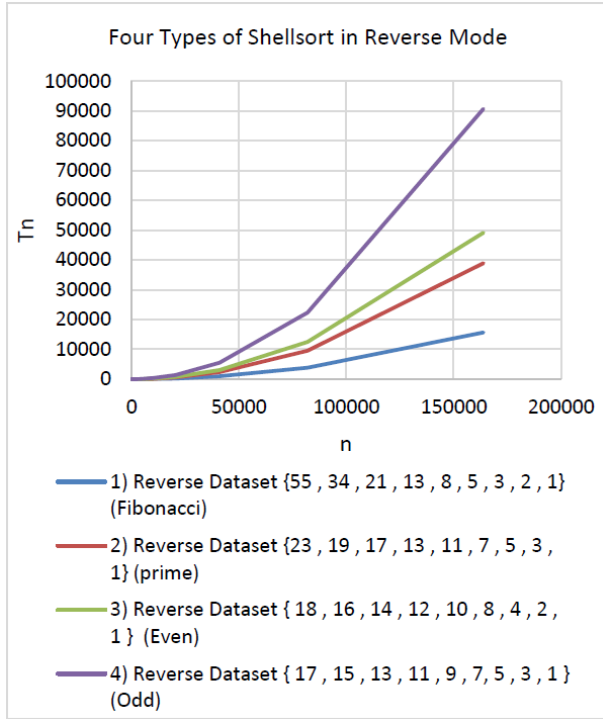


Figure 9: Four Types of Shellsort Algorithm in Reverse Mode.

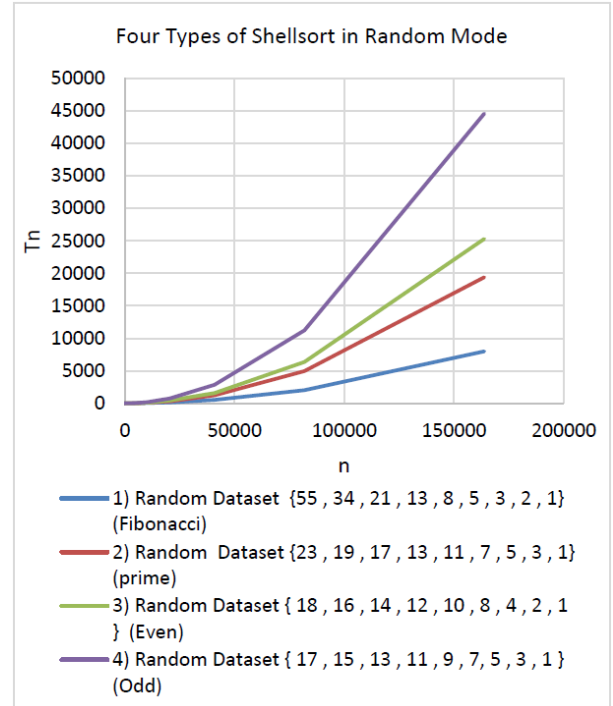


Figure 10: Four Types of Shellsort Algorithm in Random Mode.

## 6 Conclusion

This paper compares four types of algorithm and consequently implies their superiority and inferiority. We conclude that function of each algorithm depends on datatypes. The effective result in Shellsort Type 1(Fibonacci Gap-Sequences) has shown us the best gap sequences numbers are Fibonacci and Prime numbers with limited quantity. It means that the Fibonacci and Prime numbers can reduce time complexity and optimize it as is illustrated in Figure 9 and 10. We have also compared Quicksort algorithm with different pivot numbers and the result showed us pivot number, as a median or middle number of our datasets can be more efficient than other pivot modes. Other algorithms have the regular time complexity as is shown in Table I. As a future work, we plan to extend these algorithms in parallel mode and compare parallel mode with normal implementation. Subsequently, we will check their time complexity as well as the normal mode and see which algorithms will be more efficient in parallel mode than normal mode. Also, we will compare the numbers of Pivot in Quicksort algorithm to see how many pivot numbers can make Quicksort algorithms more efficient than Quicksort with one pivot number.

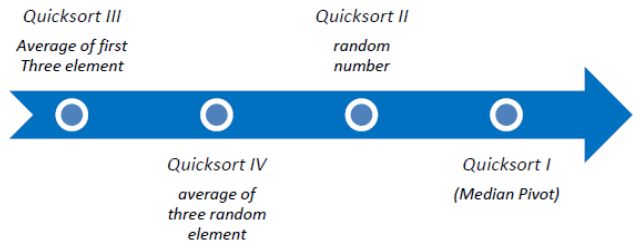


Figure 11: Quicksort Comparison based on time complexity and pivot numbers for both Reverse and Random datasets.

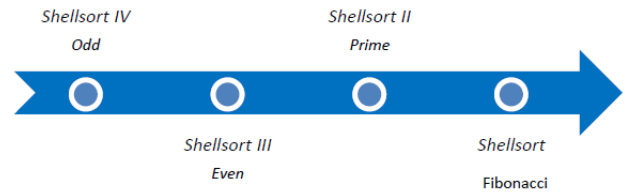
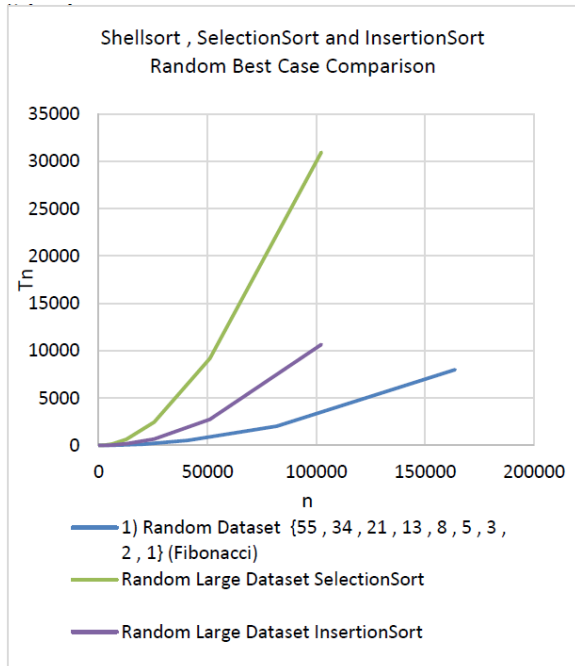
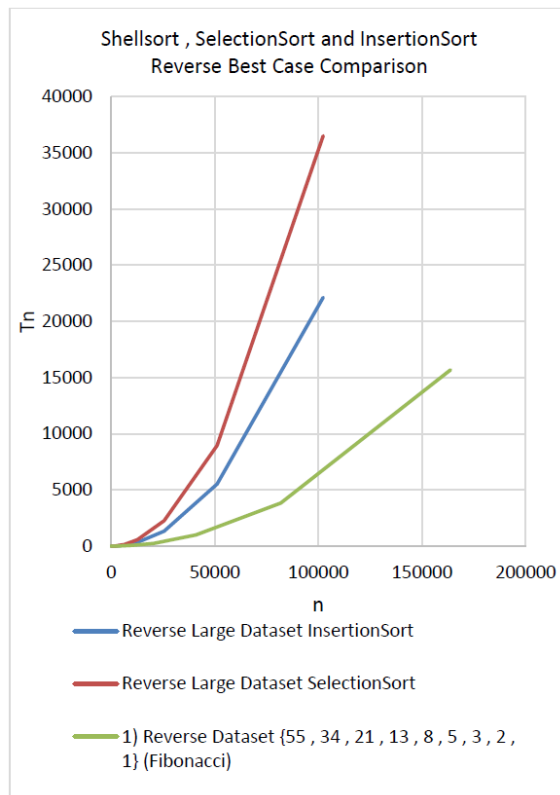


Figure 12: Shellsort Comparison based on time complexity and gap sequence sizes for both Reverse and Random datasets.



**Figure 13: Random Types of Shell Sort, Selection Sort, and Insertion Sort Comparison.**



**Figure 14: Reverse Types of Shell Sort, Selection Sort and Insertion Sort Comparison.**

## REFERENCES

- [1] Sipser, M., 2006. Introduction to the Theory of Computation (Vol. 2). Boston: Thomson Course Technology.
- [2] Hammad, Jehad. "A Comparative Study between Various Sorting Algorithms." International Journal of Computer Science and Network Security (IJCSNS), vol. 15, no. 3, (2015), pp 11-12.
- [3] Yadav, Neelam, and Sangeeta Kumari. "SORTING ALGORITHMS." International Research Journal of Engineering and Technology (IRJET), vol. 03, no. 02, (2016), pp 528-529.
- [4] Ayush Pathak, Abhijeet Vajpayee, Deepak Agrawal, "A Comparative Study of Sorting Algorithm Based on Their Time Complexity" International Journal of Engineering Sciences & Research Acropolis Institute of Technology & Research, Indore, India(IJESRT), vol. 03, no. 12, (2014), pp 629-631
- [5] Rajagopal, D., and K. Thilakavalli. "Different Sorting Algorithm's Comparison based Upon the Time Complexity." International Journal of u-and e-Service, Science and Technology (IJUNESST), vol. 9, no. 8, (2016), pp. 287-296.
- [6] Faujdar, Neetu, and Satya Prakash Ghrera. "Analysis and testing of sorting algorithms on a standard dataset." Communication Systems and Network Technologies (CSNT), 2015 Fifth International Conference on. IEEE, (2015) April, 962-967.
- [7] Aliyu, Ahmed M., and Dr PB Zirra. "A Comparative Analysis of Sorting Algorithms on Integer and Character Arrays." The International Journal of Engineering and Science (IJES), vol. 2, no. 7, (2013), pp 25-29.
- [8] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, "Introduction to Algorithms" (Third Edition) (2009).
- [9] Horowitz, Ellis, Sartaj Sahni, and Susan Anderson-Freed. Fundamentals of data structures. No. 04; QA76. D35, H6.. London: Pitman, 1983.
- [10] Bender, Michael A., Martin Farach-Colton, and Miguel A. Mosteiro. "Insertion sort is  $O(n \log n)$ ." Theory of Computing Systems 39.3 (2006): 391-397.
- [11] Shell, Donald L. "A high-speed sorting procedure." Communications of the ACM, vol. 2, no. 7, (1959), pp. 30-32.
- [12] Pratt, Vaughan R. Shellsort and Sorting Networks. No. STAN-CS-72-260. STANFORD UNIV CALIF DEPT OF COMPUTER SCIENCE, 1972.
- [13] Hoare, C. A. R. "Algorithm 64: Quicksort", vol. 4, no. 7, Comm. ACM. (1961) July, pp 321.
- [14] R. Sedgewick, "Quicksort," PhD dissertation, Stanford University, Stanford, CA, May 1975. Stanford Computer Science Report STAN-CS-75-492.
- [15] J. L. Bentley and R. Sedgewick, "Fast algorithms for sorting and Searching strings", In Proc. 8th annual ACM-SIAM Symposium on Discrete algorithms, New Orleans, Louisiana, USA, 1997, pp 360 – 369
- [16] R. Loeser, "Some performance tests of: quicksort: and descendants," Comm. ACM 17, 3, Mar. 1974, pp 143 – 152. descendants," Comm. ACM 17, 3, Mar. 1974, pp 143 – 152.
- [17] Hoare, Charles AR. "Quicksort." The Computer Journal, vol. 5, no. 1, (1962), pp. 10-16.
- [18] Horowitz, Ellis, and Sartaj Sahni. Fundamentals of computer algorithms. Computer Science Press, 1978. (PAGE 126 Quicksort).
- [19] Kumari, Anchala, Niraj Kumar Singh, and Soubhik Chakraborty. "A statistical comparative study of some sorting algorithms." International Journal in Foundations of Computer Science & Technology (IJFCST), vol. 5, no. 4, (2015), pp 28.
- [20] Jadoon, Sultanullah, Salman Faiz Solehria, and Mubashir Qayum. "Optimized selection sort algorithm is faster than insertion sort algorithm: a comparative study." International Journal of Electrical & Computer Sciences IJECS-IJENS 11.02 (2011): 19-24.
- [21] Knuth, Donald. "Sorting and Searching" Section 6.2. 1: Searching an Ordered Table, USA, vol. 3, (1997), pp. 409-426.